

Condicionales y validación

Java desde cero · Módulo 3

Tomar decisiones en un programa usando if, else if, else, operadores lógicos, switch y validaciones básicas.

IF

Objetivos del bloque

Al finalizar, el alumnado debe comprender y modificar decisiones sencillas en Java.

- Construir condiciones con if, else if y else.
- Utilizar operadores de comparación y operadores lógicos.
- Validar datos para evitar resultados absurdos o incorrectos.
- Usar switch cuando existan varias opciones cerradas.
- Leer errores básicos y saber dónde revisar.

¿Qué es una condición?

Una condición es una pregunta que Java responde con true o false.

```
int edad = 20;  
  
boolean mayorEdad = edad >= 18;  
  
System.out.println(mayorEdad);
```

La condición `edad >= 18` se evalúa. Si se cumple, el resultado es true; si no se cumple, es false.

Las condiciones permiten que el programa no haga siempre lo mismo.

Boolean: el tipo de dato de las decisiones

boolean solo puede guardar dos valores: true o false.

```
boolean tieneEntrada = true;  
boolean esMayor = false;  
  
System.out.println(tieneEntrada);  
System.out.println(esMayor);
```

true significa que una condición se cumple.
false significa que no se cumple.
El resultado de una comparación también es boolean.

Estructura básica de if

if permite ejecutar un bloque solo si la condición es verdadera.

```
int nota = 7;

if (nota >= 5) {
    System.out.println("Aprobado");
}
```

La condición va entre paréntesis.
El bloque que se ejecuta va entre llaves.
Dentro del bloque pueden ir una o varias instrucciones.

if + else

else indica qué hacer cuando la condición no se cumple.

```
int nota = 4;

if (nota >= 5) {
    System.out.println("Aprobado");
} else {
    System.out.println("Suspenso");
}
```

Solo se ejecuta uno de los dos caminos: el de if o el de else.

No se escribe condición en else, porque representa “todo lo demás”.

else if: varias posibilidades

Se usa cuando hay más de dos resultados posibles.

```
double nota = 8.2;

if (nota < 5) {
    System.out.println("Insuficiente");
} else if (nota < 7) {
    System.out.println("Bien");
} else if (nota < 9) {
    System.out.println("Notable");
} else {
    System.out.println("Sobresaliente");
}
```

Java evalúa de arriba hacia abajo.
Cuando una condición se cumple, ya no mira las siguientes.
El orden de las condiciones importa muchísimo.

Operadores de comparación

Sirven para comparar números, caracteres u otros valores.

Operador	Significado
==	igual que
!=	distinto de
>	mayor que
<	menor que
>=	mayor o igual
<=	menor o igual

```
int edad = 18;
```

```
System.out.println(edad == 18);  
System.out.println(edad >= 18);  
System.out.println(edad < 18);
```

Cuidado con = y ==

Uno asigna, el otro compara.

```
int edad = 18;    // asigna el valor 18

if (edad == 18) { // compara
    System.out.println("Tiene 18 años");
}
```

```
// MAL
if (edad = 18) {
    System.out.println("Error");
}
```

En Java, = se usa para guardar un valor. == se usa para preguntar si dos valores son iguales.

Operadores lógicos

Permiten combinar varias condiciones.

Operador	Significado	Ejemplo
&&	Y	edad >= 18 && tieneEntrada
	O	esSocio tieneInvitacion
!	NO	!bloqueado

```
int edad = 20;
boolean tieneEntrada = true;

if (edad >= 18 && tieneEntrada) {
    System.out.println("Acceso permitido");
}
```

&&: deben cumplirse todas

Con &&, todas las condiciones deben ser verdaderas.

```
int edad = 19;
boolean tieneEntrada = true;

if (edad >= 18 && tieneEntrada) {
    System.out.println("Puede entrar");
} else {
    System.out.println("No puede entrar");
}
```

Es útil cuando un acceso, descuento o permiso depende de varios requisitos a la vez.

||: basta con una condición

Con ||, se cumple si al menos una condición es verdadera.

```
boolean esSocio = false;  
boolean tieneInvitacion = true;  
  
if (esSocio || tieneInvitacion) {  
    System.out.println("Puede pasar");  
}
```

Ejemplo: puede acceder si es socio o si tiene invitación.
No hace falta que ocurran ambas cosas.

!: negar una condición

El operador ! invierte un boolean.

```
boolean bloqueado = false;

if (!bloqueado) {
    System.out.println("Usuario activo");
}
```

!true se convierte en false.

!false se convierte en true.

Conviene usarlo con nombres claros para que el código sea legible.

Validar datos

Validar significa comprobar que un dato tiene sentido antes de usarlo.

```
double nota = 12;

if (nota < 0 || nota > 10) {
    System.out.println("Nota no válida");
} else {
    System.out.println("Nota correcta");
}
```

No basta con que el código compile: el programa debe impedir resultados ilógicos.

Ejemplos de datos a validar: edad negativa, precio menor que cero, nota fuera de 0 a 10.

Ejemplo completo: clasificación de nota

Combina validación y decisiones encadenadas.

```
double nota = 8.5;

if (nota < 0 || nota > 10) {
    System.out.println("Nota no válida");
} else if (nota < 5) {
    System.out.println("Insuficiente");
} else if (nota < 6) {
    System.out.println("Suficiente");
} else if (nota < 7) {
    System.out.println("Bien");
} else if (nota < 9) {
    System.out.println("Notable");
} else {
    System.out.println("Sobresaliente");
}
```

Orden correcto de las condiciones

El orden evita que un caso quede atrapado por otro anterior.

```
// Orden correcto
if (nota < 5) {
    System.out.println("Insuficiente");
} else if (nota < 7) {
    System.out.println("Bien");
} else {
    System.out.println("Notable o sobresaliente");
}
```

```
// Orden peligroso
if (nota < 9) {
    System.out.println("Notable");
} else if (nota < 5) {
    System.out.println("Insuficiente");
}
```

Si una condición general va antes que una condición más concreta, puede impedir que la concreta llegue a ejecutarse.

switch: varias opciones cerradas

switch es cómodo cuando comparamos una variable con valores concretos.

```
int dia = 2;

String mensaje = switch (dia) {
    case 1 -> "Lunes";
    case 2 -> "Martes";
    case 3 -> "Miércoles";
    default -> "Día no válido";
};

System.out.println(mensaje);
```

Se usa con opciones concretas.
default actúa como caso por defecto.
El switch moderno usa flechas -> y es más limpio.

if o switch: ¿cuál uso?

No son enemigos: cada uno encaja mejor en situaciones diferentes.

Usa if cuando...

Hay rangos: nota ≥ 5 .

Hay condiciones compuestas: edad ≥ 18 && entrada.

Las comparaciones son diferentes entre sí.

Usa switch cuando...

Hay valores exactos: 1, 2, 3.

Hay un menú con opciones.

El código queda más claro que con muchos else if.

Entrada de datos + validación

Cuando el dato viene del usuario, hay que comprobarlo.

```
import java.util.Scanner;

public class ValidarEdad {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Edad: ");
        int edad = Integer.parseInt(sc.nextLine());

        if (edad < 0) {
            System.out.println("Edad no válida");
        } else if (edad >= 18) {
            System.out.println("Mayor de edad");
        } else {
            System.out.println("Menor de edad");
        }
    }
}
```

Errores frecuentes

Muchos errores de condicionales son pequeños detalles de sintaxis.

Error	Corrección
Falta ;	Revisar final de instrucciones
Usar = en vez de ==	= asigna; == compara
Olvidar llaves	Comprobar apertura y cierre
Condiciones mal ordenadas	Poner primero los casos más restrictivos
Comparar String con ==	Usar equals()

Comparar textos con equals()

En Java, los String se comparan con equals(), no con ==.

```
String usuario = "admin";

// Correcto
if (usuario.equals("admin")) {
    System.out.println("Acceso de administrador");
}

// Evitar
if (usuario == "admin") {
    System.out.println("Comparación incorrecta");
}
```

== compara referencias. equals() compara el contenido del texto.

Para ignorar mayúsculas y minúsculas: equalsIgnoreCase().

Buenas prácticas

Pequeñas normas para que los ejercicios sean claros y corregibles.

Usar nombres descriptivos: `nota`, `edad`, `precioTotal`.
Validar antes de calcular o mostrar un resultado definitivo.
No mezclar demasiadas ideas en un solo bloque.
Probar valores normales, extremos y erróneos.
Leer los mensajes de error antes de cambiar código al azar.

Práctica guiada 1

Aprobado o suspenso con validación sencilla.

Enunciado

Crear una variable nota. Si está fuera de 0 a 10, mostrar “Nota no válida”. Si es 5 o más, mostrar “Aprobado”. En caso contrario, mostrar “Suspenso”.

```
double nota = 6.5;

if (nota < 0 || nota > 10) {
    System.out.println("Nota no válida");
} else if (nota >= 5) {
    System.out.println("Aprobado");
} else {
    System.out.println("Suspenso");
}
```

Práctica guiada 2

Tarifa de envío.

Enunciado

Si el importe es igual o superior a 50 €, el envío es gratis. Si no, cuesta 4,99 €. Si el destino es Canarias, añade 3 € de recargo.

```
double importe = 42.00;
boolean canarias = true;

double envio;

if (importe >= 50) {
    envio = 0;
} else {
    envio = 4.99;
}

if (canarias) {
    envio = envio + 3;
}

System.out.println("Envío: " + envio);
```

Reto final del bloque

Crear un programa de clasificación y validación.

Pedir o crear una nota entre 0 y 10.

Validar que no sea menor que 0 ni mayor que 10.

Mostrar: insuficiente, suficiente, bien, notable o sobresaliente.

Probar con: -1, 0, 4.9, 5, 6, 7, 8.9, 9 y 11.

Comentar el código explicando las decisiones importantes.